

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs) 
 - Backpropagation, chain rule, gradient descent optimization
- Week 6 – Preventing Overfitting
 - Overfitting vs underfitting, regularization (L1/L2), dropout, early stopping

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - Assessment-1
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - Assessment-2
- Week 12 - Project Presentations & Discussions (Project Due)

Week-5 / Agenda

Theme: Training Neural Networks

- Theory
 - Recap - Week 4 on Neural Network Essentials
 - Training a Neural Network
 - BackPropagation Algorithm
 - Cost Functions
 - Q&A on BackPropagation and Neural Networks
- Lab Work



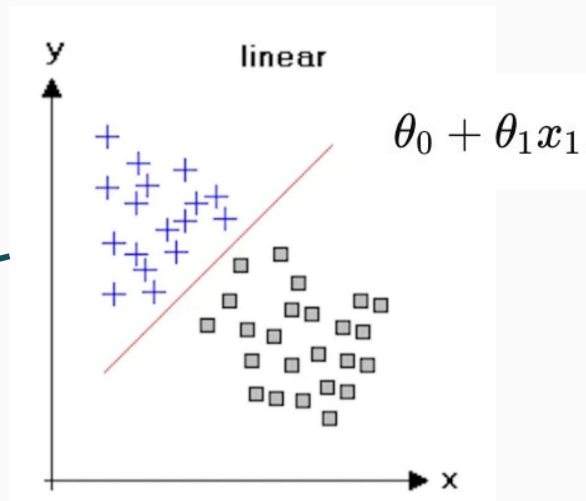
Logistic Regression Learns a Linear Boundary



And Sometimes Linear Is Enough (and Better)

Suppose you're classifying:

- Spam vs not spam
- Pass vs fail
- And the data looks like this:
 - Class A: mostly on the left
 - Class B: mostly on the right
- A straight line cleanly separates them.
- Using a nonlinear model here:
 - Adds complexity
 - Risks overfitting



Logistic Regression Learns a Linear Boundary



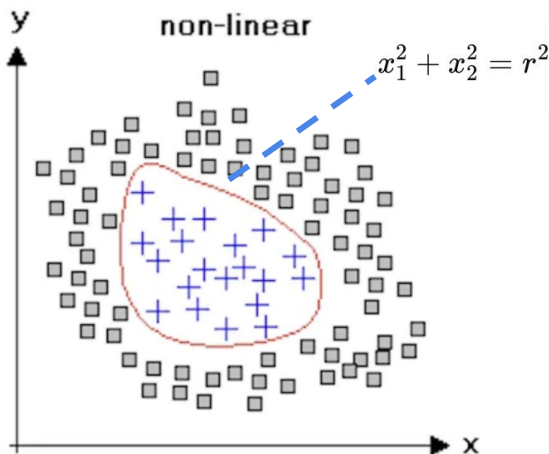
When Linear Fails

Suppose:

- Class 1 = points inside a circle
- Class 0 = points outside the circle

No straight line can separate "inside" from "outside."

- But this equation works: $x_1^2 + x_2^2 = r^2$
- That's nonlinear.
- If we add features: Logistic regression can now learn a circular boundary.



then your model becomes:

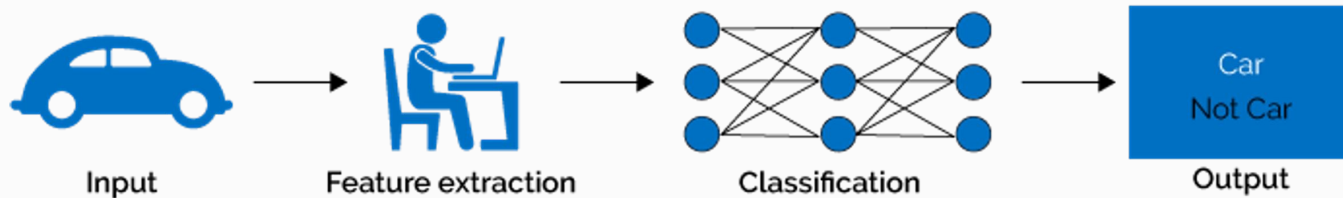
$$\theta_0 + \theta_1 z$$

which means:

$$\theta_0 + \theta_1(x_1^2 + x_2^2)$$

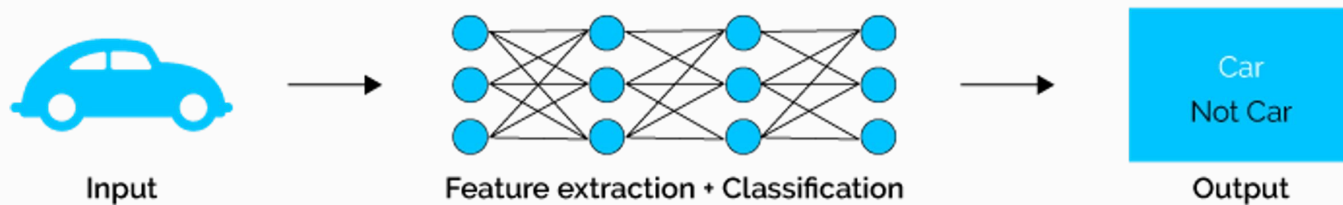
Avoiding Hand Crafted Features

Machine Learning



A neural network and hence deep learning learns internal nonlinear combinations automatically via hidden layers!!

Deep Learning



Before going further, important clarifications



DL Limitations & Cautions

- Does not eliminate all preprocessing
 - Missing value handling, scaling, encoding, sequence/window creation may still be needed
- Not universally better
 - Classical models (Random Forest, XGBoost, linear models) can outperform DL on tabular data
- Requires sufficient data and computational resources to learn effectively

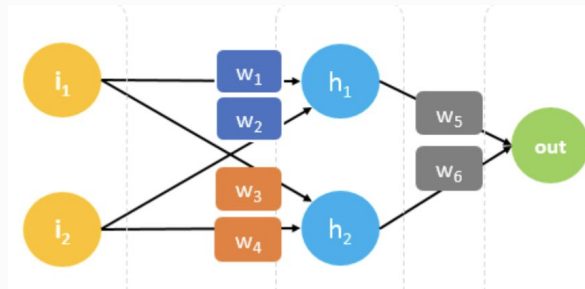
Mathematical View of a Neuron

A neuron has **three key components**:

1. **Inputs (x)**
 - Can be raw features from data or outputs from other neurons in previous layers.
 - these are the values that are fed into the neuron
2. **Weights and Bias (w,b)**
 - Weights determine how much each input contributes.
3. **Activation Function (σ)**
 - Introduces **nonlinearity**, allowing the network to model complex, nonlinear patterns.

First computes LOGIT then executes activation function

Activations are the outputs of neurons!



A neuron computes:

$$z = \sum_{i=1}^n w_i x_i + b$$

LOGIT

$$a = \sigma(z)$$

ACTIVATION

Logistic Unit (Sigmoid Neuron)



- Interpretation -

$$h_{\theta}(x) = P(Y = 1 \mid x, \theta)$$

$$z = \theta^T x$$

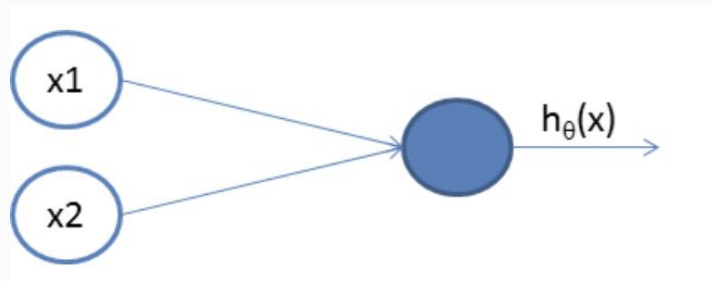
$$h_{\theta}(x) = \frac{1}{1 + e^{-z}}$$

The hypothesis function parameterized by theta, evaluated at x.

$h_{\theta}(x)$ = predicted probability that $y = 1$ given x

$h_{\theta}(x) \geq 0.5 \Rightarrow$ predict class 1

$h_{\theta}(x) < 0.5 \Rightarrow$ predict class 0

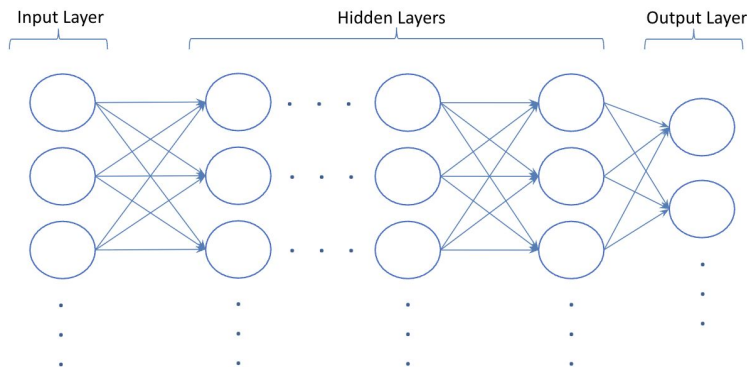




Architecture of a Neural Network

- Neural networks are built by connecting logistic units (or other types of neurons) in layers.
 - XNOR example illustrates many key principles of standard neural networks.
- There are **different types of neural network architectures**, each designed for specific kinds of data or tasks, such as:
 - **Feedforward Neural Networks (FFNNs)** – forward-only, layered networks for general tasks
 - **Convolutional Neural Networks (CNNs)** – for grid-like data such as images
 - **Recurrent Neural Networks (RNNs)** – for sequential data like time series or text
 - **Transformer Networks** – for complex sequence modeling, especially in NLP
- We will first focus on the **Feedforward Neural Network (FFNN)** !

Remember Network Architecture



Depth vs width
Notation

 Epoch = one full pass over the entire training dataset.

Iteration = one parameter update (one batch).

If you train for:

- 10 epochs
- Dataset = 10,000 samples
- Batch size = 100

Then:

- 100 iterations per epoch
- 1000 total weight updates

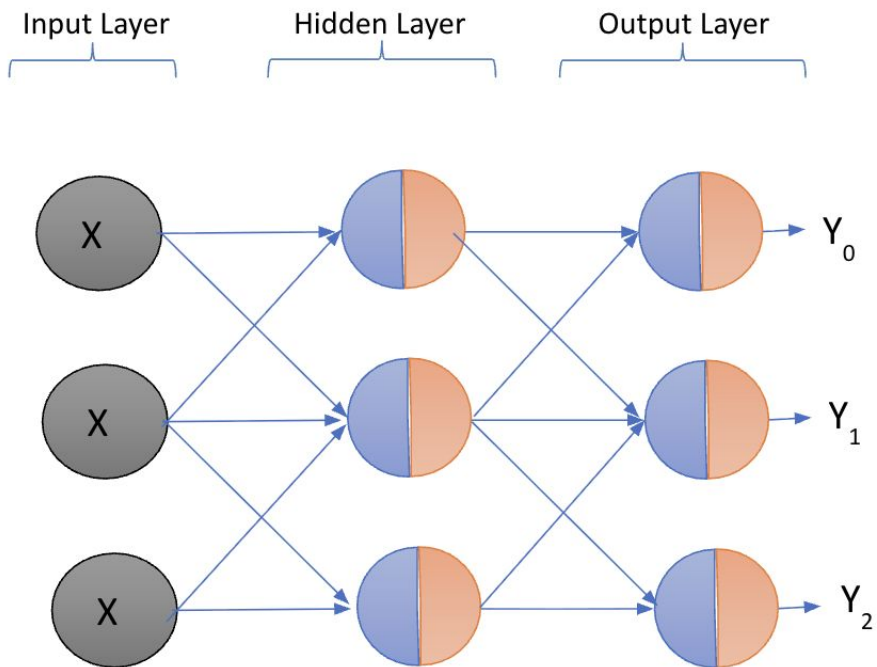
- Too few epochs → **underfitting** (model hasn't learned patterns).
- Too many epochs → **overfitting** (model memorizes training data).
- Small batches = noisy, exploratory → better generalization.
- Very large batches = smooth, predictable → may converge quickly but risk overfitting.

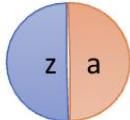
Batch size → Choose **power of 2** that fits your GPU memory and experiment.

Typical starting range: **10–100 epochs**, adjust depending on dataset and model size.

There's **no single best** – tune batch size and epochs for your dataset, model, and hardware.

Logit and Activation Functions

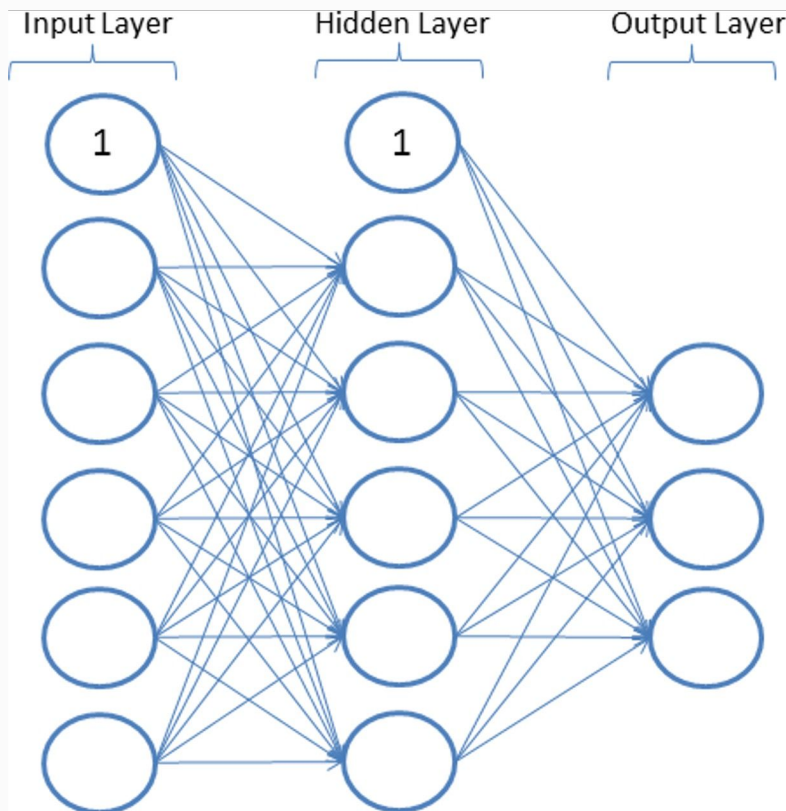


Logit  Activation Function

In the input layer there is no activation function!

The Bias Unit

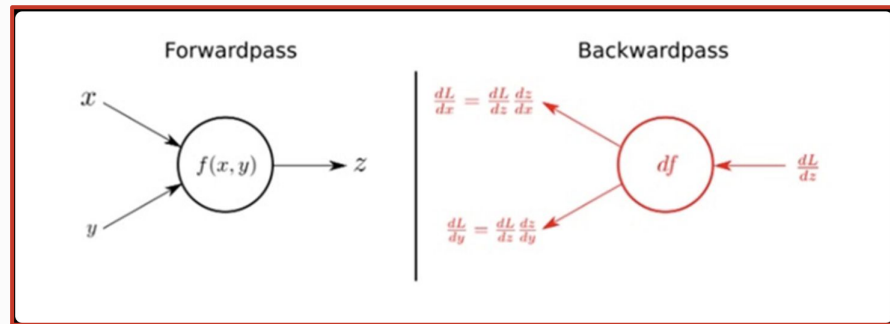
- Each layer except the output layer typically has a bias unit.
- Bias unit in layer j is not connected to units in the previous layer $j-1$!!
 - instead, it provides a fixed input of 1 to the units in the next layer.
 - Bias unit 'data' always 1
- Bias allows the neurons to shift their activation function, analogous to the intercept term in linear or logistic regression.
- Bias is often not visualised in networks!



Two Key Algorithms in Neural Networks & Deep Learning



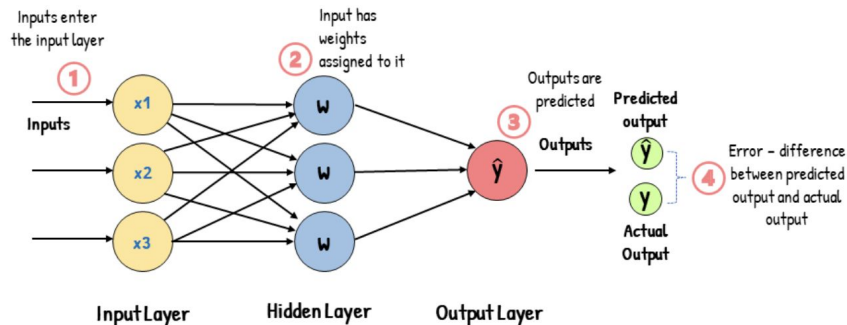
- **Forward Propagation (Feedforward Algorithm)**
 - Purpose: Compute networks current predictions for the input
 - To do so, pass information layer by layer
 - Used in training and inference
 - Does not change weights
- **Backward Propagation Algorithm**
 - Purpose: Determine how to adjust the weights to reduce prediction error.
 - Calculate gradients and provide direction for weight updates



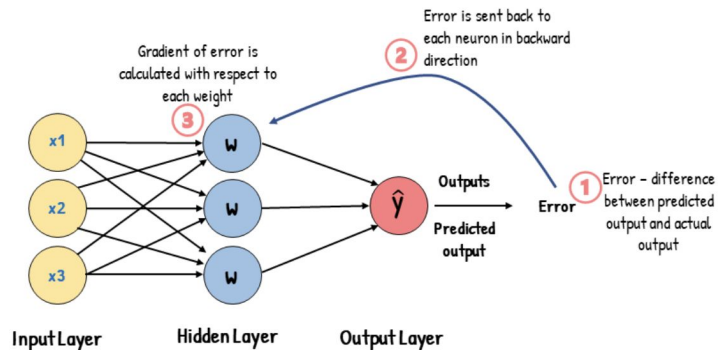
Forward pass answers: “What does the network predict?”

Backward pass answers: “How should we change the weights to improve the prediction?”

Feed-Forward Neural Network



Backpropagation



BACKPROPAGATION

What is Training a Neural Network Mean?

We want the network to **learn a function** that maps inputs x to outputs y as accurately as possible.

$$\hat{y} = f(x; W, b)$$

- The network has **parameters (weights and biases)** $\theta = \{w, b\}$
 - These are the “knobs” we can adjust.

We are learning:

- **Which weights and biases produce outputs closest to the true labels y**

Network Parameters?

- **Weights (w)** → control how strongly one neuron influences another
- **Biases (b)** → allow neurons to shift the activation function
- Together, weights + biases define **how the network processes inputs**

Learning = finding the parameter values that **minimize the loss function** over our training data.

- Mathematically:
Find θ such that $J(\theta)$ (cost) is minimized

Every training example gives feedback on how to adjust these parameters.

How do We Learn the Parameters?

- The Foundation of Learning in Neural Networks -
- **adjusting the parameters (weights and biases) based on feedback from the loss function.**
- Gradients of the parameters with respect to the loss tell us the direction and magnitude of change needed
- by repeating this process, the network gradually improves its predictions.

$$\frac{\partial J}{\partial w_{ij}}, \quad \frac{\partial J}{\partial b_i}$$

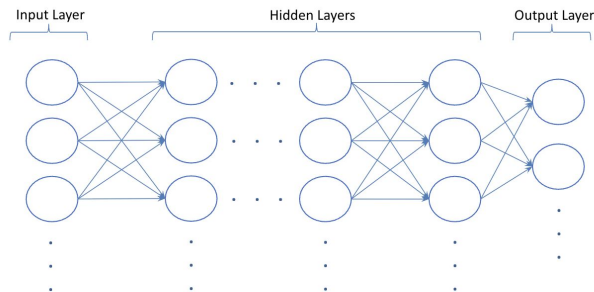
Without gradients, we wouldn't know **how to change each weight** to reduce the error.

This is why **backpropagation** (chain rule) is essential — it efficiently computes all the gradients in the network.

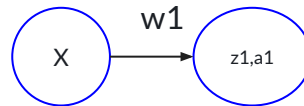
How do We Learn the Parameters?

Backpropagation = **gradient computation algorithm** for neural networks.

We use the chain rule when a function depends on a parameter indirectly through other variables.



Suppose:



$$E = E(a)$$

$$a = f(z)$$

$$z = wx$$

Now look carefully:

The loss E does **not** depend directly on w .

It depends on:

$$w \rightarrow z \rightarrow a \rightarrow E$$

So if we ask:

$$\frac{\partial E}{\partial w}$$

We cannot differentiate directly.

We must account for the whole dependency path.

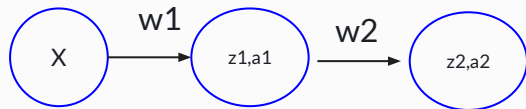
The chain rule says:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

How do We Learn the Parameters?

Backpropagation = **gradient computation algorithm** for neural networks.

We use the chain rule when a function depends on a parameter **indirectly through other variables**.



Toy example:

- 1 input
- 1 hidden neuron
- 1 output neuron

When computing a weight's gradient, chain only follows the path through activations and logits, all other weights along the way are treated as constants.

the chain is exactly:

$$w^{(1)} \rightarrow z^{(1)} \rightarrow a^{(1)} \rightarrow z^{(2)} \rightarrow a^{(2)} \rightarrow E$$

Applying the Chain Rule

$$\frac{\partial E}{\partial w^{(1)}} = \frac{\partial E}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

We calculate Delta / Error Signal in Backpropagation for each node

1. What it is:

- Delta (δ) = the **error signal** for a neuron.
- It tells us **how much that neuron contributed to the total error**.
- Formally:

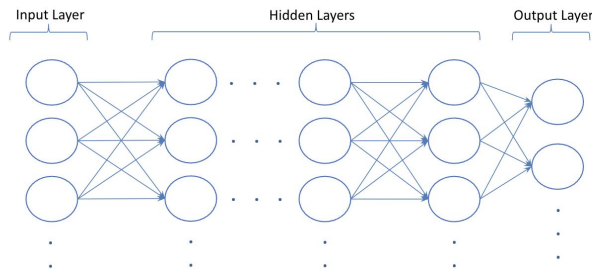
$$\delta_j = \frac{\partial E_{total}}{\partial z_j}$$

where z_j is the neuron's input (logit) and E_{total} is the loss.

2. Why we need it:

- Backpropagation is about **propagating error backward**.
- To update a weight w_{ij} connecting neuron $i \rightarrow j$, we use:

$$\frac{\partial E_{total}}{\partial w_{ij}} = \delta_j \cdot a_i$$



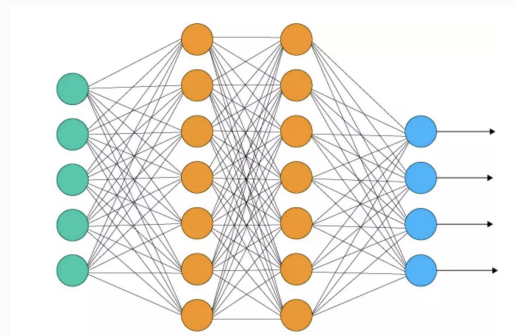
$$z_j = \sum_k w_{kj} a_k + b_j$$
$$\frac{\partial z_j}{\partial w_{ij}} = a_i$$

- You **must calculate δ for every neuron** during backpropagation, because each one affects how we update the weights feeding into it.
- Once you have δ for a neuron, **updating all incoming weights is just $\delta \times$ input activations**.

Backpropagation

Backpropagation = a systematic way to **calculate gradients for all network parameters efficiently**, uses **dynamic programming under the hood to reuse intermediate results**

- Why we need it:
 - a. Directly computing derivatives for deep networks is messy
 - b. Backprop applies the **chain rule layer by layer**
 - c. Computes exactly how each weight/bias affects the final loss
- Once we have gradients → we can **update the parameters** with gradient descent



General parameter update rule

$$\theta := \theta - \eta \frac{\partial J}{\partial \theta}$$

- θ = **any parameter** in the network (weight w or bias b)
- η = learning rate (step size)
- $\frac{\partial J}{\partial \theta}$ = **gradient of the loss w.r.t. that parameter**

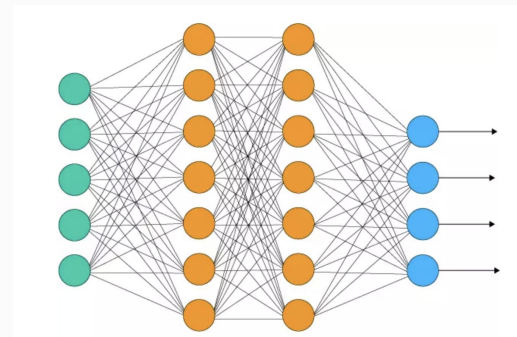
Gradient Calculation in Neural Network

Loss depends on all weights at the same time

$$J = J(w_1, w_2, \dots, w_n)$$

- The weights do not affect the loss directly.
- They affect neurons → neurons affect other neurons → eventually affect the output → which affects the loss.

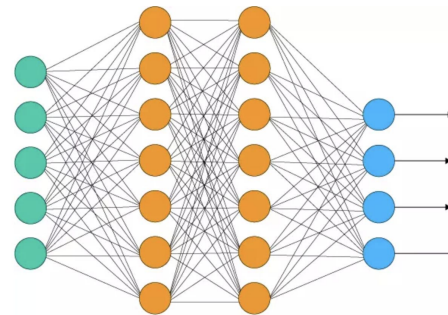
So every parameter influences the loss **through other parameters and activations**.



Backpropagation

Backpropagation answers this question:

“If I slightly change this weight, how much will the final loss change?”



- Even though weights affect each other through layers, we can measure their individual contribution to the final error.
- Individual contribution of a weight to the error is the **gradient of the loss with respect to that weight**, therefore the **partial derivative of loss with respect to that weight**.

- Backpropagation computes for every weight w_i

$$\frac{\partial L}{\partial w_i}$$

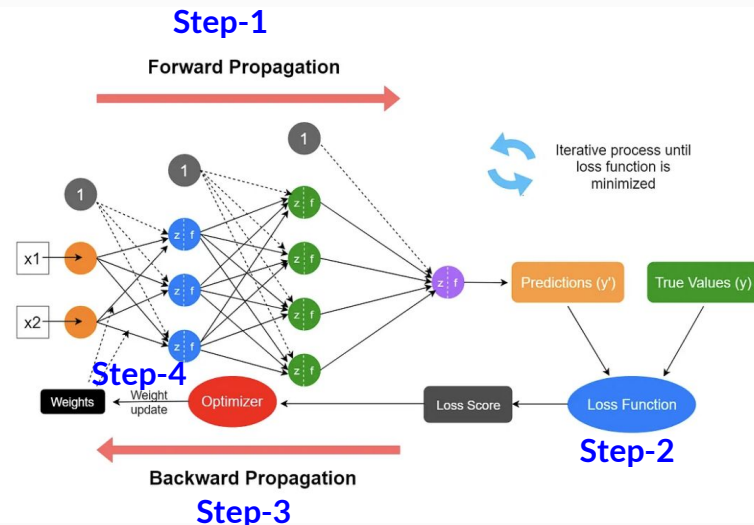
Training a Neural Network

Training of a neural network is an iterative process in which the calculations are carried out **forward** and **backward through each layer** in the network *until the loss function is minimized*.

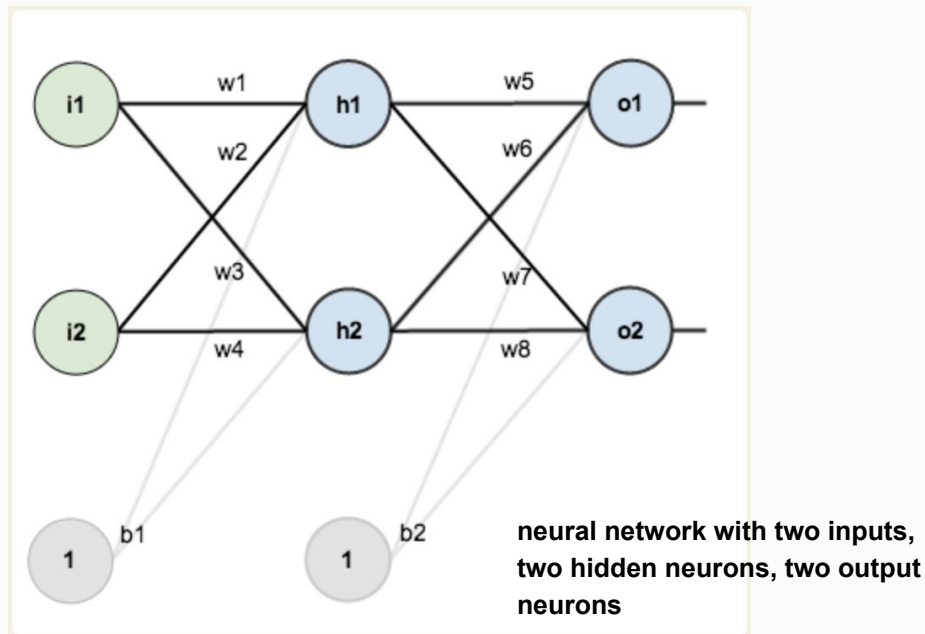
Training a neural network involves repeating 4 steps:

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. Backward Pass (**Backpropagation**) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.



Training and BackPropagation Step by Step



Our initial weights will be as following:

$w1 = 0.15$, $w2 = 0.20$, $w3 = 0.25$, $w4 = 0.30$,
 $w5 = 0.40$, $w6 = 0.45$, $w7 = 0.50$, $w8 = 0.55$
 $b1 = 0.35$, $b2 = 0.60$

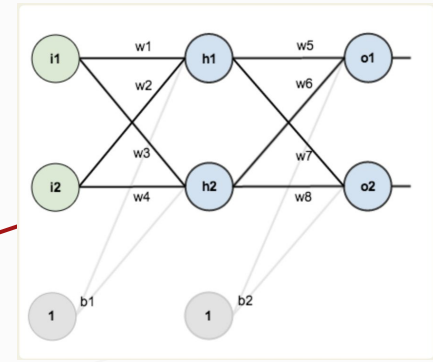
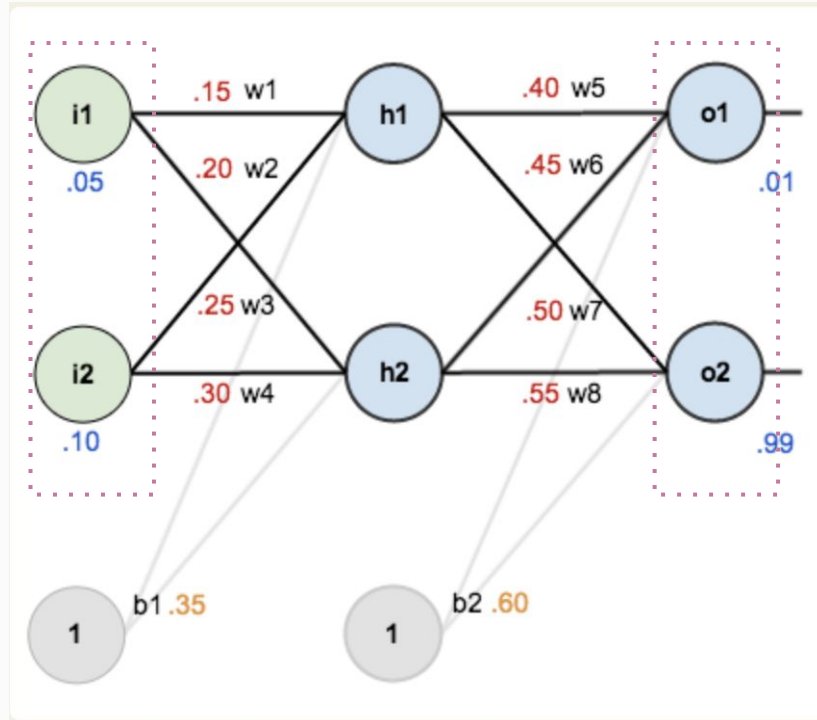
TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. Backward Pass (**Backpropagation**) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.

- Neural network training is about finding weights that minimize prediction error.
- We usually start our training with a set of randomly generated weights.
- Then use backpropagation to update the weights in an attempt to correctly map inputs to outputs.

Backpropagation Step by Step



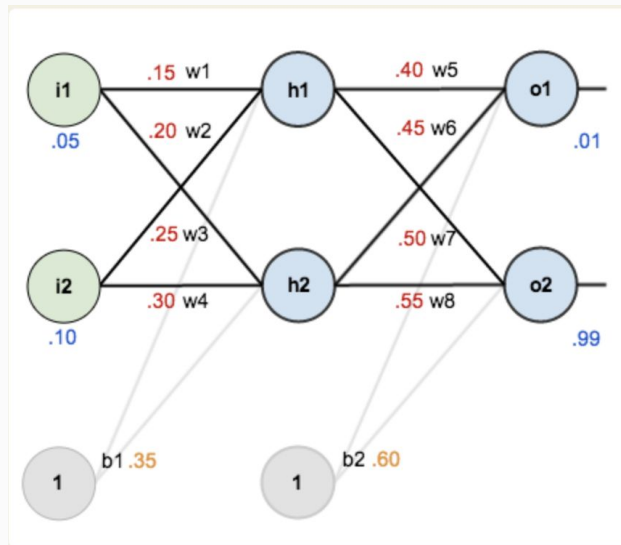
Our initial weights will be as following:
 $w_1 = 0.15$, $w_2 = 0.20$, $w_3 = 0.25$, $w_4 = 0.30$,
 $w_5 = 0.40$, $w_6 = 0.45$, $w_7 = 0.50$, $w_8 = 0.55$
 $b_1 = 0.35$, $b_2 = 0.60$

Dataset

We're going to work with a single training set:
given inputs **0.05 and 0.10**, we want the neural network to output **0.01 and 0.99**.

STEP 1: Forward Pass/

Compute Predictions/FeedForward Alg)



TRAINING A NEURAL NETWORK

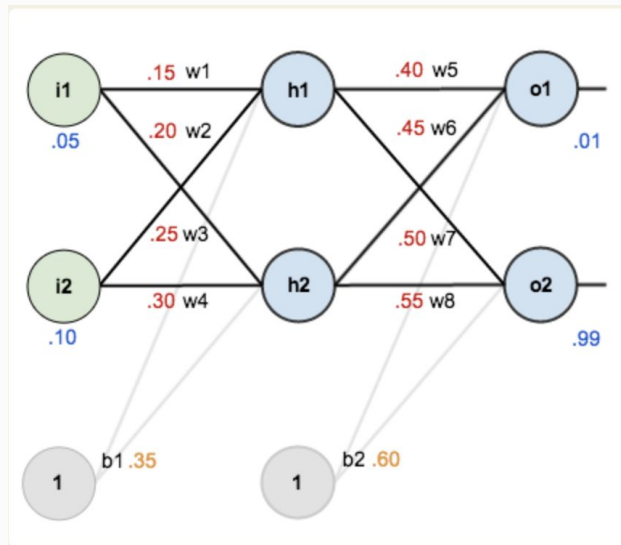
1. **Forward Pass** → **Compute predictions**
2. Compute Loss → Measure error
3. Backward Pass (**Backpropagation**) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.

- let's see what network **currently** predicts given the weights and biases and inputs of 0.05 and 0.10 .
- To do this we'll feed those inputs forward through the network.
 - We use given weights and inputs to predict the output.
 - Inputs are multiplied by weights
 - Results are then passed forward to next layer.
- We will use **sigmoid as activation function throughout** the network for keeping things simple.

STEP 1: Forward Pass/

Compute Predictions/FeedForward Alg)



Step 1 — Hidden Layer

First compute the pre-activation (logit) for h_1 :

$$z_{h1} = w_1 i_1 + w_2 i_2 + b_1$$

$$z_{h1} = 0.15 \cdot 0.05 + 0.20 \cdot 0.10 + 0.35$$

$$z_{h1} = 0.3775$$

Now apply the sigmoid activation:

$$a_{h1} = \sigma(z_{h1}) = \frac{1}{1 + e^{-z_{h1}}}$$

$$a_{h1} = \frac{1}{1 + e^{-0.3775}} = 0.593269992$$

Repeating the same process for h_2 :

$$a_{h2} = 0.596884378$$

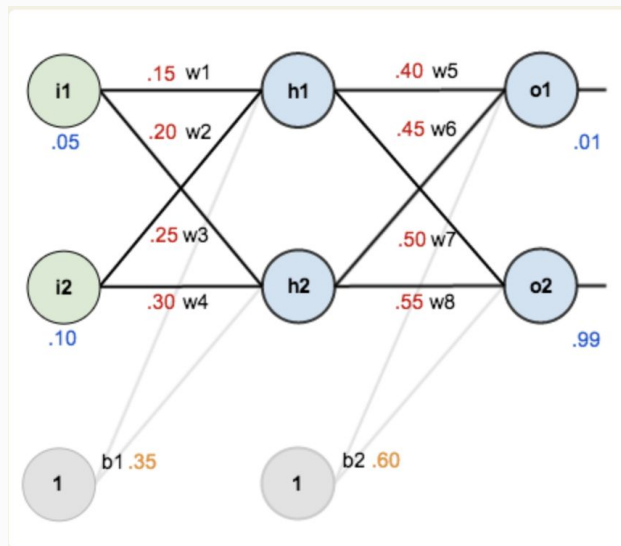
- z = weighted sum + bias (pre-activation, logit)
- a = activation value (neuron output)
- Forward pass = compute $z \rightarrow$ apply activation $\rightarrow$ pass a forward

TRAINING A NEURAL NETWORK

1. **Forward Pass $\rightarrow$ Compute predictions**
2. Compute Loss $\rightarrow$ Measure error
3. Backward Pass (**Backpropagation**) $\rightarrow$ Compute gradients
4. Update Weights $\rightarrow$ Reduce error

Repeat until convergence.

STEP 1: Forward Pass/Output Layer Activation Values



Step 2 — Output Layer

Now compute the pre-activation for o_1 :

$$z_{o1} = w_5 a_{h1} + w_6 a_{h2} + b_2$$

$$z_{o1} = 0.40 \cdot 0.593269992 + 0.45 \cdot 0.596884378 + 0.60$$

$$z_{o1} = 1.105905967$$

Apply the sigmoid activation:

$$a_{o1} = \sigma(z_{o1}) = \frac{1}{1 + e^{-z_{o1}}}$$

$$a_{o1} = \frac{1}{1 + e^{-1.105905967}} = 0.75136507$$

where target
 $o1=0.1$

Repeating the same process for o_2 :

$$a_{o2} = 0.772928465$$

where target
 $o2=0.99$

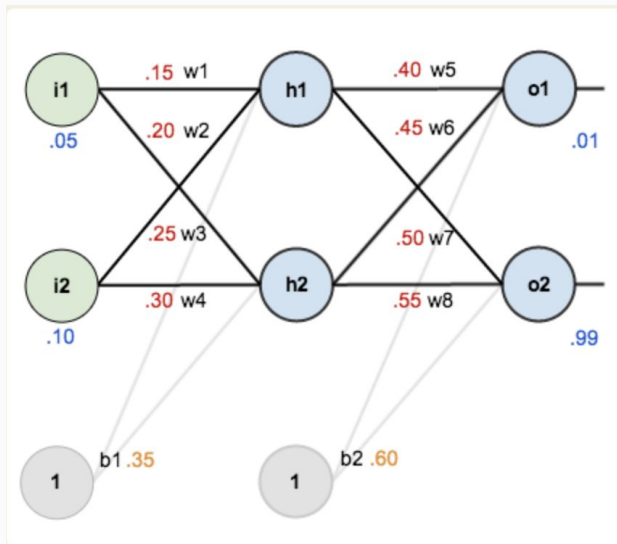
- z = weighted sum + bias (pre-activation, logit)
- a = activation value (neuron output)
- Forward pass = compute $z \rightarrow$ apply activation $\rightarrow$ pass a forward

TRAINING A NEURAL NETWORK

1. **Forward Pass $\rightarrow$ Compute predictions**
2. Compute Loss $\rightarrow$ Measure error
3. Backward Pass (**Backpropagation**) $\rightarrow$ Compute gradients
4. Update Weights $\rightarrow$ Reduce error

Repeat until convergence.

Calculating the Loss



We now compute the error at the output layer using the **squared error (MSE) loss**:

$$E_{total} = \sum \frac{1}{2} (t - a)^2$$

where

- t = target (ideal output)
- a = network output (activation)

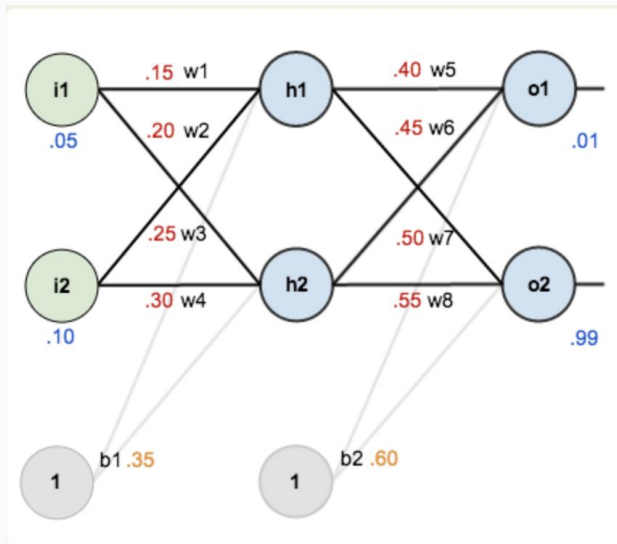
TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. **Compute Loss → Measure error**
3. Backward Pass (Backpropagation) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.

- Now, it's time to find out how our network performed by calculating the difference between actual output and predicted one.
- We use MSE as the loss function

Calculating the Loss



TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. **Compute Loss → Measure error**
3. Backward Pass (Backpropagation) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.

- Now, it's time to find out how our network performed by calculating the difference between actual output and predicted one.
- We use MSE as the loss function

Error for Output Neuron o1: $t_{o1} = 0.01$
 Network output: $a_{o1} = 0.75136507$

Error →

$$E_{o1} = \frac{1}{2}(t_{o1} - a_{o1})^2$$

$$E_{o1} = \frac{1}{2}(0.01 - 0.75136507)^2$$

$$E_{o1} = 0.274811083$$

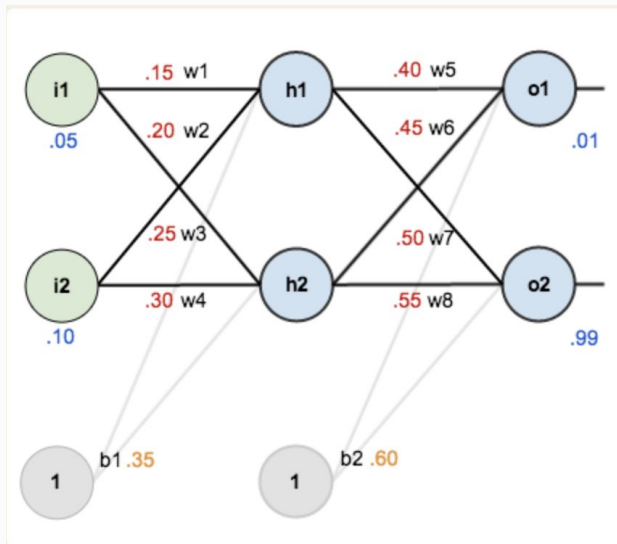
Error for Output Neuron o2: $t_{o2} = 0.99$
 Network output: $a_{o2} = 0.772928465$

Error →

$$E_{o2} = \frac{1}{2}(t_{o2} - a_{o2})^2$$

$$E_{o2} = 0.023560026$$

Calculating the Loss



Error for Output Neuron o1: $t_{o1} = 0.01$
Network output: $a_{o1} = 0.75136507$

Error →

$$E_{o1} = \frac{1}{2}(t_{o1} - a_{o1})^2$$

$$E_{o1} = \frac{1}{2}(0.01 - 0.75136507)^2$$

$$E_{o1} = 0.274811083$$

Error for Output Neuron o2: $t_{o2} = 0.99$
Network output: $a_{o2} = 0.772928465$

Error →

$$E_{o2} = \frac{1}{2}(t_{o2} - a_{o2})^2$$

$$E_{o2} = 0.023560026$$

Total Network Error

$$E_{total} = E_{o1} + E_{o2}$$

$$E_{total} = 0.274811083 + 0.023560026$$

$$E_{total} = 0.298371109$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. **Compute Loss → Measure error**
3. Backward Pass (Backpropagation) → Compute gradients
4. Update Weights → Reduce error

Repeat until convergence.

Reducing the Error

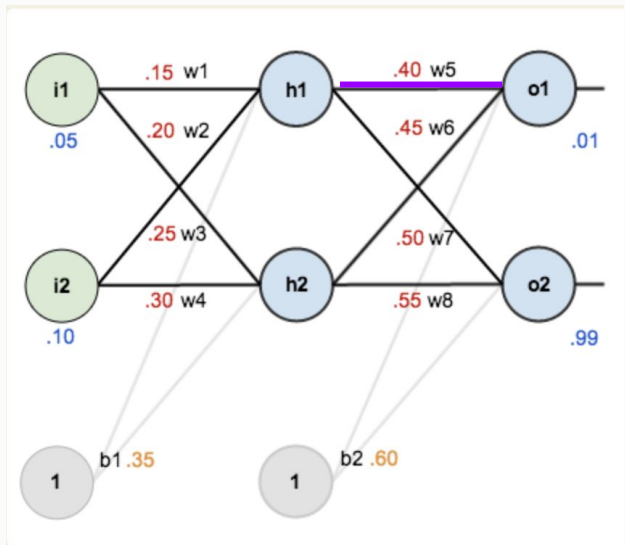
- Our main goal of training is to reduce error which is the difference between prediction and actual output.
- To do so, we will adjust the weights in a way that the next time the error/loss should be less!
- **Backpropagation** calculates the gradient of the error function with respect to the neural network weights.
- Gradient gives us the amount and direction of change for each weight; w_1 , w_2 , w_3 , w_4 , w_5 and w_6 , etc.
- Calculation proceeds backwards through the network.

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

BackPropagation - Output Layer



TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

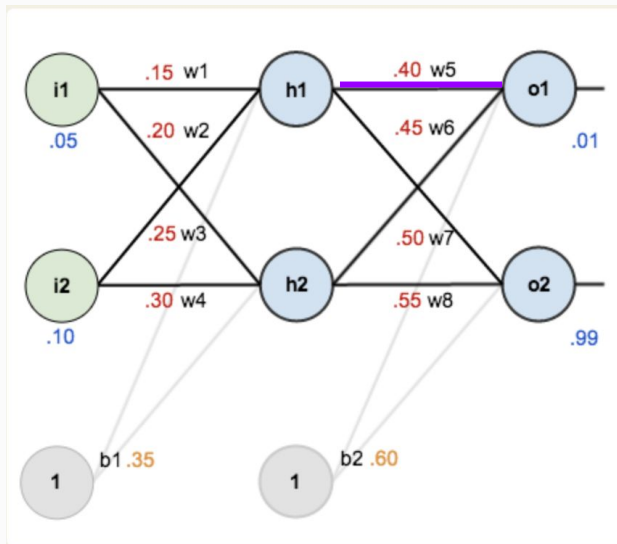
Example - Updating w5

We want to compute:

$$\frac{\partial E_{total}}{\partial w_5}$$

- This tells us how much a small change in w5 affects the total error.
- It is read as:
 - partial derivative of E_{total} with respect to w5
 - Or
 - The gradient with respect to w5

BackPropagation - Output Layer



TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

Step 1 — Apply the Chain Rule

Since w_5 affects E_{total} through a_{o1} and z_{o1} , we apply the chain rule:

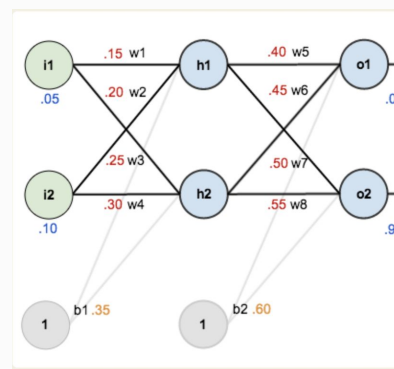
$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}} \cdot \frac{\partial z_{o1}}{\partial w_5}$$

We now compute each term separately.

BackPropagation - Output Layer

$$\frac{\partial E_{total}}{\partial w_5} = \underbrace{\frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}}}_{\delta} \cdot \underbrace{\frac{\partial z_{o1}}{\partial w_5}}_{a_i}$$

This part is the input from h1



TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

Step 2 — Derivative of Error w.r.t Output

Total error:

$$E_{total} = \frac{1}{2}(t_{o1} - a_{o1})^2 + \frac{1}{2}(t_{o2} - a_{o2})^2$$

When differentiating with respect to a_{o1} , the second term becomes zero (since it does not depend on a_{o1}).

$$\frac{\partial E_{total}}{\partial a_{o1}} = -(t_{o1} - a_{o1})$$

Substitute values:

$$= -(0.01 - 0.75136507) = 0.74136507$$

Step 3 — Derivative of Activation (Sigmoid)

Sigmoid:

$$a_{o1} = \sigma(z_{o1})$$

Derivative:

$$\frac{\partial a_{o1}}{\partial z_{o1}} = a_{o1}(1 - a_{o1})$$

$$= 0.75136507(1 - 0.75136507) = 0.186815602$$

Step 4 — Derivative of Pre-Activation w.r.t Weight

Input travels through the weight to the output node

$$z_{o1} = w_5 a_{h1} + w_6 a_{h2} + b_2$$

$$\frac{\partial z_{o1}}{\partial w_5} = a_{h1} = 0.593269992$$

Step 5 — Combine Everything

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 \cdot 0.186815602 \cdot 0.593269992 = 0.082167041$$

BackPropagation - Output Layer

Step 5 — Combine Everything

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 \cdot 0.186815602 \cdot 0.593269992 = 0.082167041$$

To reduce the error:

$$w_5^{new} = w_5 - \eta \frac{\partial E_{total}}{\partial w_5}$$

Using $\eta = 0.5$:

$$w_5^{new} = 0.4 - 0.5 \cdot 0.082167041 = 0.35891648$$

Updated Weights

Repeating this process:

$$w_6^{new} = 0.408666186$$

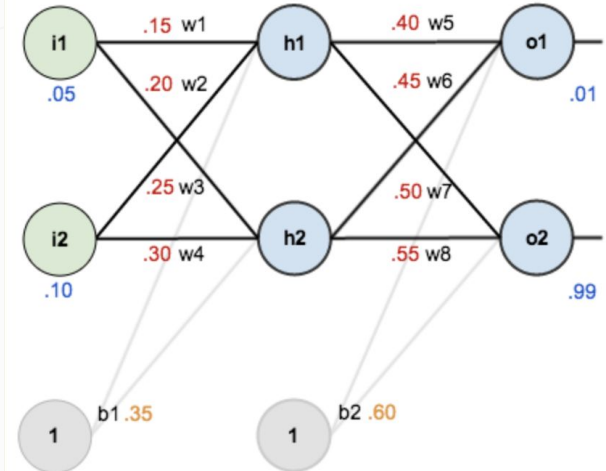
$$w_7^{new} = 0.511301270$$

$$w_8^{new} = 0.561370121$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. Backward Pass (Backpropagation) → Compute gradients
4. **Update Weights → Reduce error**

Repeat until convergence.



Delta Notation - Output Layer/**General** Gradient Formula

$$\frac{\partial E_{total}}{\partial w_5} = \underbrace{\frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}}}_{\delta} \cdot \underbrace{\frac{\partial z_{o1}}{\partial w_5}}_{a_i}$$

This part is the input from h1

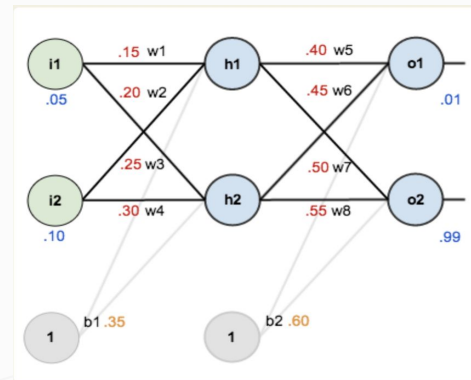
δ

Delta (error signal) tells us how changing the input to the neuron (z) affects the total error.

- It tells us how much the neuron contributed to the total error.
 - If δ is large $\rightarrow$ neuron is "responsible" for a big part of the error $\rightarrow$ its weights need bigger updates
 - If δ is small $\rightarrow$ neuron is only slightly responsible $\rightarrow$ smaller updates

a_i

We multiply by the source neuron's activation because weight scales its input — the bigger the input, the more a change in the weight affects the neuron's logit, and therefore the total error.



Delta Notation - Output Layer/General Gradient Formula

$$\frac{\partial E_{total}}{\partial w_5} = \underbrace{\frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}}}_{\delta} \cdot \underbrace{\frac{\partial z_{o1}}{\partial w_5}}_{a_{h1}}$$

This part is the input from h1

For a weight w_{ij} connecting:

source neuron $i \rightarrow$ destination neuron j

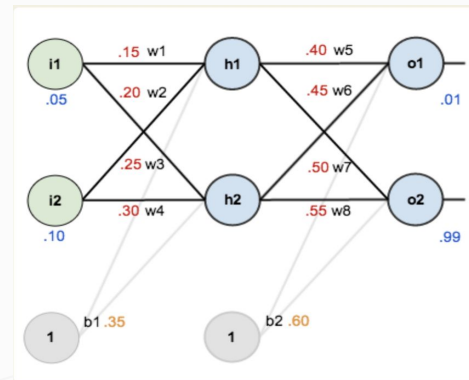
We have:

For any weight, the gradient equals:
destination node's delta $\times$ source node's

$$\frac{\partial E}{\partial w_i} = \delta \cdot \text{input} \quad \longrightarrow \quad \frac{\partial E}{\partial w_{ij}} = \delta_j \cdot a_i$$

Where:

- $\delta_j = \frac{\partial E}{\partial z_j}$
(error signal at the destination node)
- a_i = activation of the source node



Delta (error signal) tells us how changing the input to the neuron (z) affects the total error.

BackPropagation - Output Layer

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}} \cdot \frac{\partial z_{o1}}{\partial w_5}$$

This part is the input from h1

$$\frac{\partial E_{total}}{\partial a_{o1}} = -(t_{o1} - a_{o1})$$

$$\frac{\partial a_{o1}}{\partial z_{o1}} = a_{o1}(1 - a_{o1})$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

Delta Notation (More Compact Form)

We define the **output node delta**:

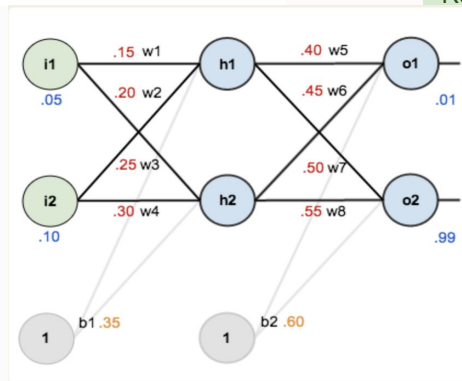
$$\delta_{o1} = \frac{\partial E_{total}}{\partial z_{o1}} = \frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}}$$

$$\delta_{o1} = -(t_{o1} - a_{o1}) \cdot a_{o1}(1 - a_{o1})$$

Then:

$$\frac{\partial E_{total}}{\partial w_5} = \delta_{o1} \cdot a_{h1}$$

(Some sources absorb the negative sign into δ .)



- delta tells us, how much the total error changes if the pre-activation (z) changes slightly?
- Delta measures how “responsible” a neuron is for the total error.
- Delta separates the “error part” from the “input part”

What to Remember?

The Gradient Has Two Core Parts

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial a_{o1}} \cdot \frac{\partial a_{o1}}{\partial z_{o1}} \cdot \frac{\partial z_{o1}}{\partial w_5}$$

For every neuron:

$$\frac{\partial E}{\partial w} = \delta \cdot \text{input}$$

Error signal × Local sensitivity = Delta

Delta × Input = Gradient

Local Sensitivity/Activation Derivative

- Measures how sensitive the neuron output is
- Local property of the neuron
- Depends only on its activation function

$$\frac{\partial a}{\partial z}$$

Error Signal

- how responsible neuron is for the final error

$$\frac{\partial E_{total}}{\partial a_{o1}}$$

BackPropagation - Hidden Layer

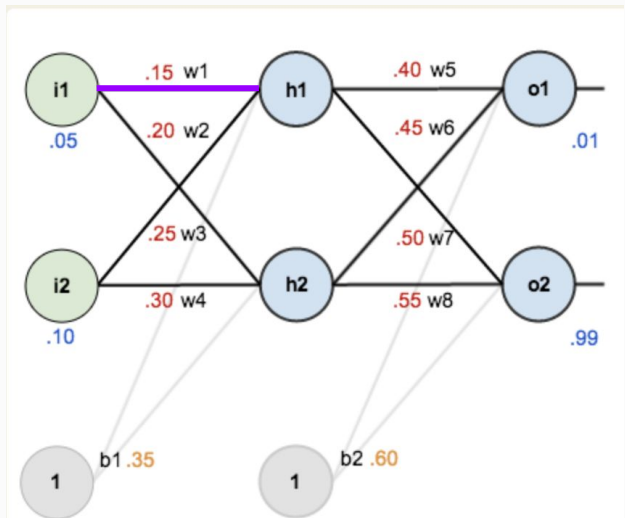
General Idea

- Hidden layers receive their error signal **from the next layer**.
 - They inherit responsibility from the neurons they influence.

error signal on node h × activation of node i

$$\frac{\partial E}{\partial w_{ih}} = \delta_h \cdot a_i$$

w_{ij} connects node_i → node_j



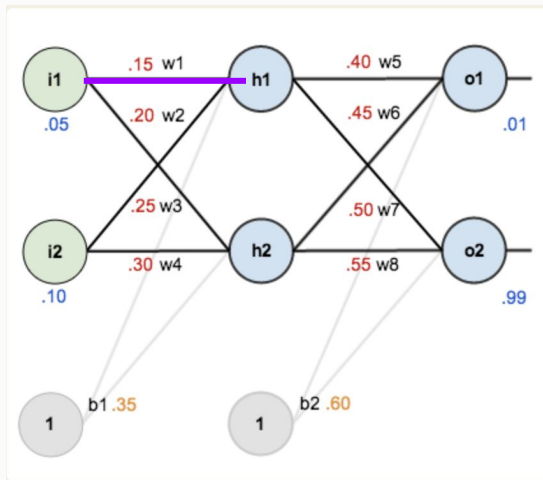
The hidden neuron's error signal is:

$$\delta_h = \left(\sum_o \delta_o \cdot w_{ho} \right) \cdot \frac{\partial a_h}{\partial z_h}$$

Sum of error signals from all neurons in the next layer that are connected to it

Sensitivity of the activation of that neuron

BackPropagation - Hidden Layer



Contribution from o_1

We already computed:

$$\frac{\partial E_{o1}}{\partial a_{h1}} = \frac{\partial E_{o1}}{\partial z_{o1}} \cdot \frac{\partial z_{o1}}{\partial a_{h1}}$$

$$\frac{\partial E_{o1}}{\partial z_{o1}} = 0.138498562$$

Since:

$$z_{o1} = w_5 a_{h1} + w_6 a_{h2} + b_2$$

$$\frac{\partial z_{o1}}{\partial a_{h1}} = w_5 = 0.40$$

Therefore:

$$\frac{\partial E_{o1}}{\partial a_{h1}} = 0.138498562 \cdot 0.40 = 0.055399425$$

Contribution from o_2

Following the same process:

$$\frac{\partial E_{o2}}{\partial a_{h1}} = -0.019049119$$

Total Effect on Hidden Neuron

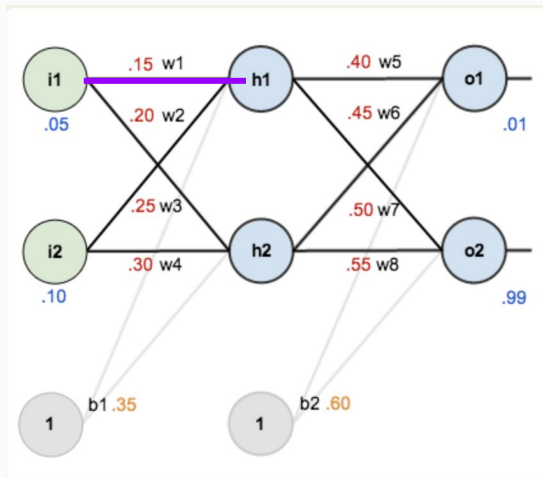
$$\frac{\partial E_{total}}{\partial a_{h1}} = 0.055399425 + (-0.019049119) = 0.036350306$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

BackPropagation - Hidden Layer



Step 2 — Derivative of Hidden Activation

Sigmoid derivative:

$$\begin{aligned}\frac{\partial a_{h1}}{\partial z_{h1}} &= a_{h1}(1 - a_{h1}) \\ &= 0.59326999(1 - 0.59326999) = 0.241300709\end{aligned}$$

Step 3 — Derivative w.r.t Weight

$$z_{h1} = w_1 i_1 + w_3 i_2 + b_1$$

$$\begin{aligned}\frac{\partial z_{h1}}{\partial w_1} &= i_1 = 0.05\end{aligned}$$

Step 4 — Combine Everything

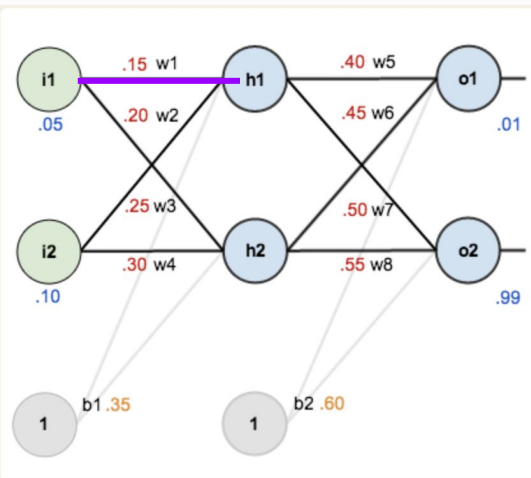
$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 \cdot 0.241300709 \cdot 0.05 = 0.000438568$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. **Backward Pass (Backpropagation) → Compute gradients**
4. Update Weights → Reduce error

Repeat until convergence.

BackPropagation - Hidden Layer



Step 5 — Update the Weight

$$w_1^{new} = w_1 - \eta \frac{\partial E_{total}}{\partial w_1}$$

Using $\eta = 0.5$:

$$w_1^{new} = 0.15 - 0.5 \cdot 0.000438568 = 0.149780716$$

Updated Hidden Weights

$$w_2^{new} = 0.19956143$$

$$w_3^{new} = 0.24975114$$

$$w_4^{new} = 0.29950229$$

TRAINING A NEURAL NETWORK

1. Forward Pass → Compute predictions
2. Compute Loss → Measure error
3. Backward Pass (Backpropagation) → Compute gradients
4. **Update Weights → Reduce error**

Repeat until convergence.

Gradient of a weight w_{ij} (general rule)

For any weight connecting:

**This formula is the same
for all neurons!**

- Source neuron $i \rightarrow$ Destination neuron j

$$\frac{\partial E}{\partial w_{ij}} = \delta_j \cdot a_i$$

Where:

- a_i = activation of source neuron (input to the weight)
- $\delta_j = \frac{\partial E}{\partial z_j}$ = delta of destination neuron

Output neuron j

$$\delta_j = \frac{\partial E}{\partial z_j} = (a_j - y_j) \cdot g'(z_j)$$

- a_j = output neuron activation
- y_j = target value
- $g'(z_j)$ = derivative of activation (sigmoid sensitivity)
- Here, delta is **directly from the error**.

Hidden neuron j

$$\delta_j = \frac{\partial E}{\partial z_j} = g'(z_j) \cdot \sum_k (\delta_k \cdot w_{jk})$$

- Sum over all neurons k in the next layer
- w_{jk} = weights from this hidden neuron to downstream neurons
- $g'(z_j)$ = activation derivative (sensitivity)
- Here, delta is **inherited from downstream errors**.

Do I need to know the derivations?

- In short, no.
- However, you should have an understanding of the terminology and ideas as presented in these slides.
- You can find full derivation in the notebooks.

What to Remember

Big Picture Pattern

Backprop always follows this structure:

Error flows backward



Each neuron computes δ



Each weight gradient = $\delta \times \text{input}$

Backpropagation works by sending error signals backward, computing δ for each neuron, and updating each weight using: $\text{delta} \times \text{input}$.

- Backpropagation is a systematic way to distribute total cost backward.
- To summarize, backprop:
 1. Computes output error
 2. Moves backward layer by layer
 3. Reuses previously computed gradients (**efficient!**) → memoization
 - a. When computing gradients layer by layer from output → input, **backprop reuses intermediate results** instead of recalculating them multiple times.
 4. Updates each weight based on its true contribution

Behind the Scenes

1. Precompute the formulas

- a. Once the **network architecture is fixed** (number of layers, neurons, and connections):
 - i. **formulas for the gradients** (how each weight and bias affects the loss) are determined by the structure and the chain rule.
 - ii. For example, in our network, the gradient formula for w_1 is same structure-wise, meaning for a network with the same layout

2. Each training iteration

1. Feed the **inputs** forward → compute **predictions**
2. Compute **intermediate values** like $h_1, h_2, (\text{prediction} - \text{actual})$, etc, all values required to calculate gradients
3. Plug these **numerical values** into the **precomputed gradient formulas**
4. Update **weights and biases** using the gradients

Backpropagation Algorithm

Step 1: Initialize weights and learning rate

- Start with **random weights** so the network isn't symmetric (otherwise all neurons learn the same thing).
- Choose a **learning rate η** to control how big each update step is.
- This is preparation before any training begins.

Step 2: Forward pass

- Take a **training input** and pass it through the network **layer by layer**.
 1. Multiply inputs by weights.
 2. Sum them (weighted sum = pre-activation z).
 3. Apply **activation function** (like sigmoid) $\rightarrow$ get output a .
 4. Pass these activations to the next layer.
- Repeat until you get the **final network output**.
- Intuition: "See what the network predicts given current weights."

Step 3: Backpropagation of error

- Compute δ for output units:

$$\delta_k = (a_k - y_k) \cdot g'(z_k)$$

- Measures how much each output neuron contributed to the error.
- Compute δ for hidden units (**layer by layer backward**):

$$\delta_j = g'(z_j) \cdot \sum_k w_{jk} \delta_k^{\text{next layer}}$$

- Hidden neurons don't have direct targets, so their δ is **backpropagated** from the next layer.
- Intuition: "Assign blame to all neurons for the final output error."

Step 4: Update weights

- For each weight w_{ij} connecting neuron i to neuron j :

$$w_{ij} \leftarrow w_{ij} - \eta \cdot \delta_j \cdot a_i$$

- For biases, previous activation $a_i = 1$.
- Intuition: "Adjust weights proportionally to how much they contributed to the error."

Loss Functions for Neural Networks

Where Is Loss Used?

The loss function:

- Quantifies error
- Provides gradients (via backprop)
- Guides weight updates

Loss → Gradients → Weight Updates

During training:

1. Forward pass → compute prediction
2. **Compute loss → compare prediction vs target**
3. Backpropagation → compute gradients of loss
4. Gradient descent → update weights

Neural Networks: Cost Functions & Multiclass Outputs

Mean Squared Error (MSE)

$$E = \frac{1}{2} \sum (y - a)^2$$

- Works well for regression.

Problem with if we use it for classification through using sigmoid outputs:

- Sigmoid output: $a = \frac{1}{1+e^{-z}} \in [0, 1]$
- MSE: $E = \frac{1}{2}(y - a)^2$

Gradient w.r.t weight:

$$\frac{\partial E}{\partial w} = (a - y) \cdot a(1 - a) \cdot x$$

- Problem: if $a \approx 0$ or $a \approx 1$, $a(1 - a) \approx 0 \rightarrow$ gradient tiny $\rightarrow$ **learning slows**.
- So, when prediction is very wrong (say $y = 1$, $a = 0.01$) $\rightarrow$ small gradient $\rightarrow$ network struggles to correct itself quickly.

Neural Networks: Cost Functions & Multiclass Outputs

Mean Squared Error (MSE)

$$E = \frac{1}{2} \sum (y - a)^2$$

- Works well for regression.
- Problem with if we use it for classification through using sigmoid outputs:

- Sigmoid output: $a = \frac{1}{1+e^{-z}} \in [0, 1]$
- MSE: $E = \frac{1}{2}(y - a)^2$

Gradient w.r.t weight:

$$\frac{\partial E}{\partial w} = (a - y) \cdot a(1 - a) \cdot x$$

This is similar to the **vanishing gradient problem**: small gradients $\rightarrow$ slow learning.

- Problem: if $a \approx 0$ or $a \approx 1$, $a(1 - a) \approx 0 \rightarrow$ gradient tiny $\rightarrow$ **learning slows**.
- So, when prediction is very wrong (say $y = 1$, $a = 0.01$) $\rightarrow$ small gradient $\rightarrow$ network struggles to correct itself quickly.

From MSE to Cross-Entropy for Classification

Better Loss for Classification: Cross-Entropy Loss (CELF)

- Designed for **probabilities** in $[0,1]$
- Binary Cross-Entropy:

$$E = -(y \log(a) + (1 - y) \log(1 - a))$$

- Gradient:

$$\frac{\partial E}{\partial w} = (a - y) \cdot x$$

- No extra $a(1-a)$ term $\rightarrow$ gradient stays meaningful even for extreme outputs
- Faster and more stable learning for classification tasks

Multivariate Outputs / Multi-Class Classification

- Binary targets → single output neuron.
- Multiclass targets → use **1-of-K / one-hot encoding** → multiple output neurons.
- Example: 3-class classification → 3 output neurons
- You can take the class with the highest output/probability

Problem with multiple independent sigmoids

- Each neuron outputs a probability, but they **may not sum to 1**.
- Using `max()` to pick the largest output seems fine, but:
 - Max function derivative = 0 almost everywhere → breaks backprop.
- Sigmoid could produce **outputs very close to 0 or 1**
 - If you use **Cross-Entropy loss** with exact 0 or 1 → $\log(0)$ occurs → **undefined / NaN**
 - Training becomes unstable

Softmax solves all these problems

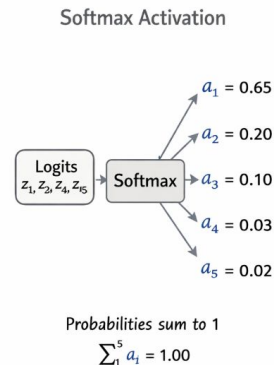
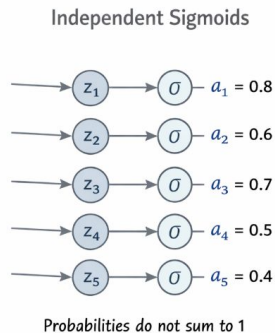
Softmax Activation

For a vector of logits $\mathbf{z} = [z_1, z_2, \dots, z_L]$, the softmax output for the i -th neuron is:

$$a_i = \frac{e^{z_i}}{\sum_{j=1}^L e^{z_j}}$$

Where:

- $z_i \rightarrow$ the **logit** (input to the softmax) for neuron i
- $L \rightarrow$ total number of output neurons
- $\sum_{j=1}^L e^{z_j} \rightarrow$ sum of exponentials of all logits



- Each a_i is a **probability**: $0 \leq a_i \leq 1$
- The **sum of all a_i is 1** $\rightarrow$ proper probability distribution

Softmax Activation

- Converts logits into a **clean probability distribution** that sums to 1
- Example for same logits $[2, 1, 0]$:

$$a_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \Rightarrow [0.71, 0.26, 0.03]$$

- Now only **one neuron dominates**, others are smaller → clean prediction
- Derivatives are well-defined → **backprop works**

Best Combinations: Activation + Loss Functions

Regression

Output activation

Linear activation

$$a = z$$

Loss function

Mean Squared Error (MSE)

$$L = (y - \hat{y})^2$$

Used for

- House prices
- Temperature prediction
- Continuous values

Why it works

- Output can be any real number

Binary Classification

Output activation

Sigmoid

$$a = \frac{1}{1 + e^{-z}}$$

Loss function

Binary Cross-Entropy

$$L = -[y \log a + (1 - y) \log(1 - a)]$$

Used for

- Spam detection
- Yes/No prediction
- Fraud detection

Why it works

- Output becomes probability (0–1)

Best Combinations: Activation + Loss Functions

Multi Class Classification

Output activation

Softmax

$$a_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

Loss function

Categorical Cross-Entropy

$$L = - \sum y_i \log a_i$$

Used for

- Image classification
- Digit recognition
- Language classification

Why it works

- Outputs sum to 1
- Interpretable probabilities

Hidden Layers - Most Common Activation Function

Activation

ReLU

$$a = \max(0, z)$$

Why ReLU?

- Simple
- Fast
- Avoids vanishing gradients
- Works well in deep networks

Best Combinations: Activation + Loss Functions

Problem	Output Activation	Loss Function
Regression	Linear	MSE
Binary classification	Sigmoid	Binary Cross-Entropy
Multi-class classification	Softmax	Cross-Entropy

- Good combinations simplify backpropagation.
- Example: Sigmoid + Cross-Entropy gives:

$$\delta = a - y$$

- instead of a complicated expression. This makes training faster and more stable.

Announcements

- Project Proposals are due today
 - Not graded
 - To provide feedback early on
- *First Assessment is on 11th of March*
 - You should be in the class for the assessment.

Lab Work

- Please follow the instructions in the Week-5 notebook in the last cell to build a notebook from scratch and play with some vector operations.

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

- Backpropagation itself is just a way to compute the gradients of the loss with respect to every weight and bias in the network.
- Gradient Descent is a separate step that uses those gradients to actually update the weights.

2. Why do we need to calculate error signals (δ) in backpropagation?

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

- Backpropagation itself is just a way to compute the gradients of the loss with respect to every weight and bias in the network.
- Gradient Descent is a separate step that uses those gradients to actually update the weights.

2. Why do we need to calculate error signals (δ) in backpropagation?

- Error signals measures how much each neuron contributes to the total error.
- Without δ , we wouldn't know which weights to adjust or by how much.

3. What impact does backpropagation have on the input layer?

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

- Backpropagation itself is just a way to compute the gradients of the loss with respect to every weight and bias in the network.
- Gradient Descent is a separate step that uses those gradients to actually update the weights.

2. Why do we need to calculate error signals (δ) in backpropagation?

- Error signals measures how much each neuron contributes to the total error.
- Without δ , we wouldn't know which weights to adjust or by how much.

3. What impact does backpropagation have on the input layer?

- The input layer itself is not updated because its activations come directly from the data.
- However, input activations affect errors in hidden layers and thus indirectly influence weight updates in the first hidden layer.

4. What does it mean if a weight after training has a value close to 0?

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

- Backpropagation itself is just a way to compute the gradients of the loss with respect to every weight and bias in the network.
- Gradient Descent is a separate step that uses those gradients to actually update the weights.

2. Why do we need to calculate error signals (δ) in backpropagation?

- Error signals measures how much each neuron contributes to the total error.
- Without δ , we wouldn't know which weights to adjust or by how much.

3. What impact does backpropagation have on the input layer?

- The input layer itself is not updated because its activations come directly from the data.
- However, input activations affect errors in hidden layers and thus indirectly influence weight updates in the first hidden layer.

4. What does it mean if a weight after training has a value close to 0?

- That connection contributes very little to the output, i.e., the input feature or path is not important for the task.
- Could be pruned or considered negligible.

5. How does backpropagation interact with network biases?

Backpropagation Questions

1. What is the relationship between Gradient Descent and Backpropagation?

- Backpropagation itself is just a way to compute the gradients of the loss with respect to every weight and bias in the network.
- Gradient Descent is a separate step that uses those gradients to actually update the weights.

2. Why do we need to calculate error signals (δ) in backpropagation?

- Error signals measures how much each neuron contributes to the total error.
- Without δ , we wouldn't know which weights to adjust or by how much.

3. What impact does backpropagation have on the input layer?

- The input layer itself is not updated because its activations come directly from the data.
- However, input activations affect errors in hidden layers and thus indirectly influence weight updates in the first hidden layer.

4. What does it mean if a weight after training has a value close to 0?

- That connection contributes very little to the output, i.e., the input feature or path is not important for the task.
- Could be pruned or considered negligible.

5. How does backpropagation interact with network biases?

- Biases have no inputs, but they affect all downstream activations.
- Backprop calculates error signals for neurons affected by biases, and weights for biases are updated like normal weights

Additional Questions

1. For a given input, do we always compute the same output?

Additional Questions

1. For a given input, do we always compute the same output?
 - Yes, once the network is trained and the parameters (weights and biases) are fixed, the same input will always produce the same output.

Additional Questions

1. For a given input, do we always compute the same output?
 - Yes, once the network is trained and the parameters (weights and biases) are fixed, the same input will always produce the same output.
2. What are the parameters of the network?

Additional Questions

1. For a given input, do we always compute the same output?
 - Yes, once the network is trained and the parameters (weights and biases) are fixed, the same input will always produce the same output.
2. What are the parameters of the network?
 - Parameters of a network: the weights and biases that the network learns during training.
 - Weights (W): The connection strengths between neurons.
 - Biases (b): The offsets added to each neuron's input.

Additional Questions

1. For a given input, do we always compute the same output?
 - Yes, once the network is trained and the parameters (weights and biases) are fixed, the same input will always produce the same output.
2. What are the parameters of the network?
 - Parameters of a network: the weights and biases that the network learns during training.
 - Weights (W): The connection strengths between neurons.
 - Biases (b): The offsets added to each neuron's input.
3. What are the hyperparameters of the network?

Additional Questions

3. What are the hyperparameters of the network?
- settings chosen before training that control learning, such as:
 - i. learning rate: How big the steps are when updating weights.
 - ii. number of layers/neurons: Controls network capacity.
 - iii. batch size
 - iv. activation functions: How neurons “fire” (ReLU, sigmoid, etc.).
 - v. regularization

Regularization

Techniques to **prevent overfitting** (when the network memorizes training data but performs poorly on new data).

Common types:

L1/L2 regularization: Adds a penalty to large weights.

Dropout: Randomly ignores some neurons during training.

Early stopping: Stop training when validation performance stops improving.

Batch size

During training, we don't always use the full dataset at once.

Batch size = number of training examples processed **before updating the network weights**.

Example: Dataset has 1,000 samples. Batch size = 100 → the network updates weights **after every 100 samples**, not after all 1,000.

Why: Speeds up training and smooths gradient updates.

usually, **batches are chosen randomly** from the training data.

Additional Questions

4. Given a layer of the network where the weights are all $= 0$, what can we say about the logit values of the following layer?

Additional Questions

4. Given a layer of the network where the weights are all = 0, what can we say about the logit values of the following layer?

- For a layer computing logits:

$$z = W \cdot x + b$$

- If **all weights** $W = 0$, this simplifies to:

$$z = 0 \cdot x + b = b$$

- Each logit in this layer is exactly equal to its bias, independent of the input.
- If biases are also 0, then all logits = 0.
- A layer with zero weights isn't "dead"; it's just temporarily inactive until training updates the weights.
- During forward pass: outputs are constant (biases).
- During backpropagation: gradients can still flow through this layer, so weights can start updating from zero.
- Until weights change from zero, the layer cannot learn meaningful input-output mappings.

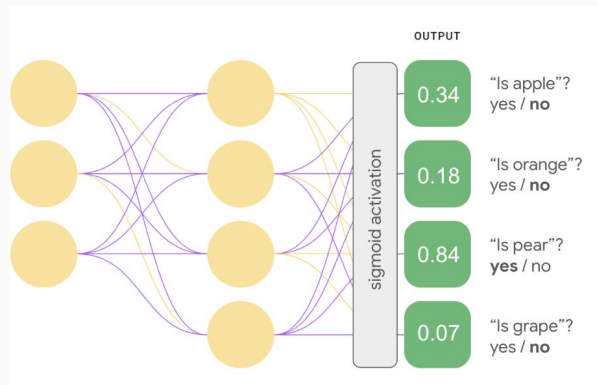
Additional Questions

5. Draw out an example feedforward network with more outputs than inputs?
Why might a network like this make sense?

Additional Questions

5. Draw out an example feedforward network with more outputs than inputs?
Why might a network like this make sense?

- Why more outputs than inputs might make sense:
 - Multi-class classification: Input features are fewer than classes (e.g., 2 features, 10 classes → 10 outputs).
 - Encoding multiple predictions: Each output could predict a different property of the same input.
 - Feature expansion: Sometimes outputs are higher-dimensional embeddings than inputs to capture more expressive information.



APPENDIX